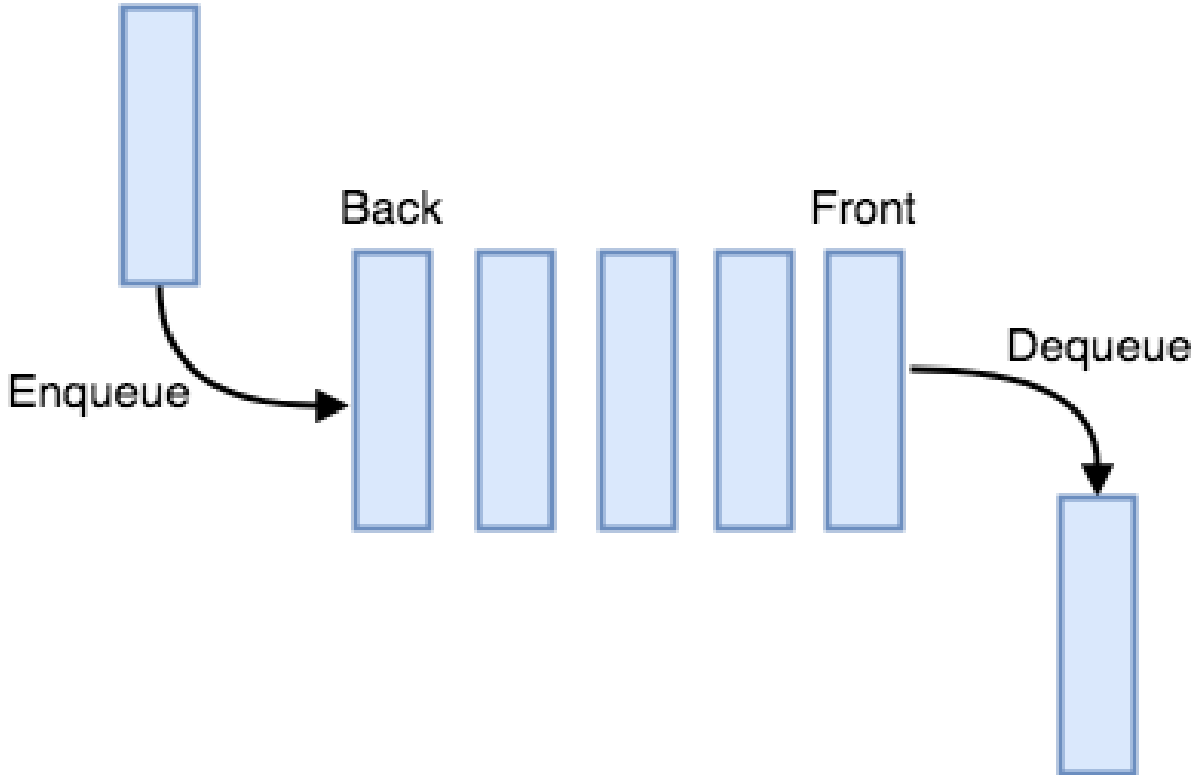
Queue Data Structure

By

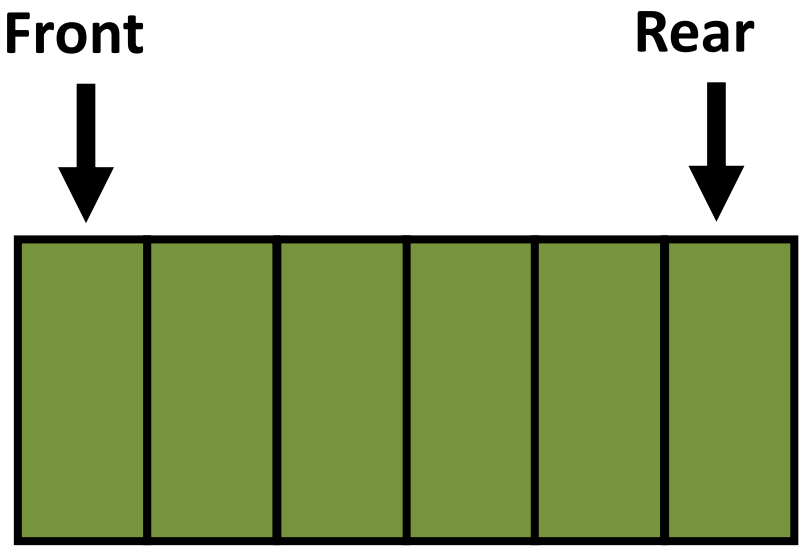
Aurora Computer Studies | auroracs.lk



What is Queue Data Structure?

Queue is a **First in First out (FIFO)** data structure.

- It is linear data structure.
- Operations are done only at front and rear of the queue.



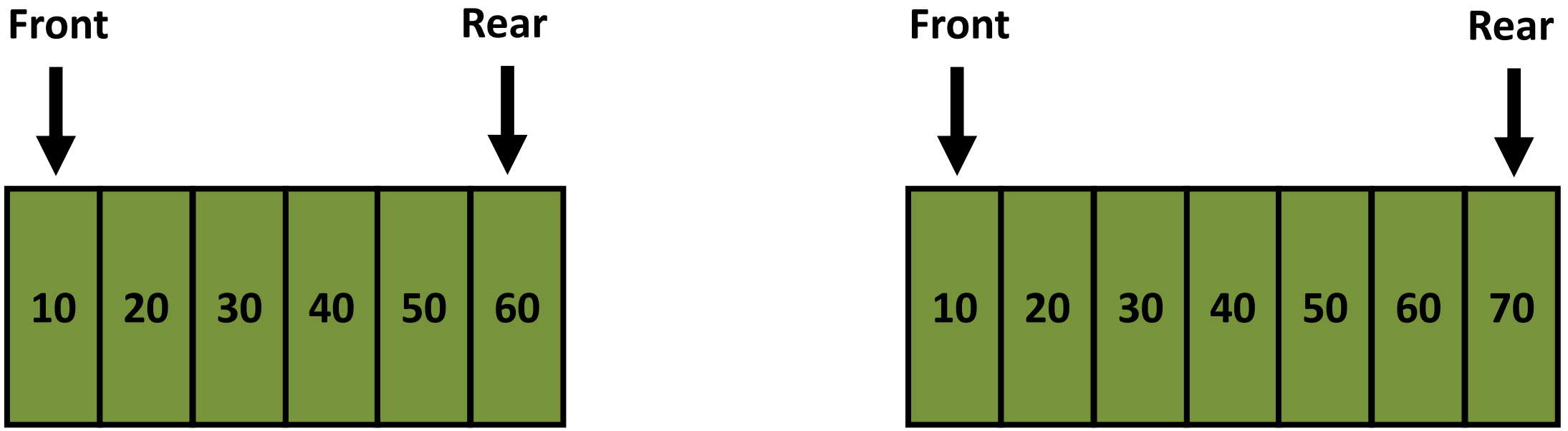
Queue - Usages

- Operating systems schedule jobs.
- Simulation of real-world queues such as lines at a ticket counter or any other first-come first-served scenario requires a queue.
- Asynchronous data transfer (file IO, pipes, sockets).
- Indirect Applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

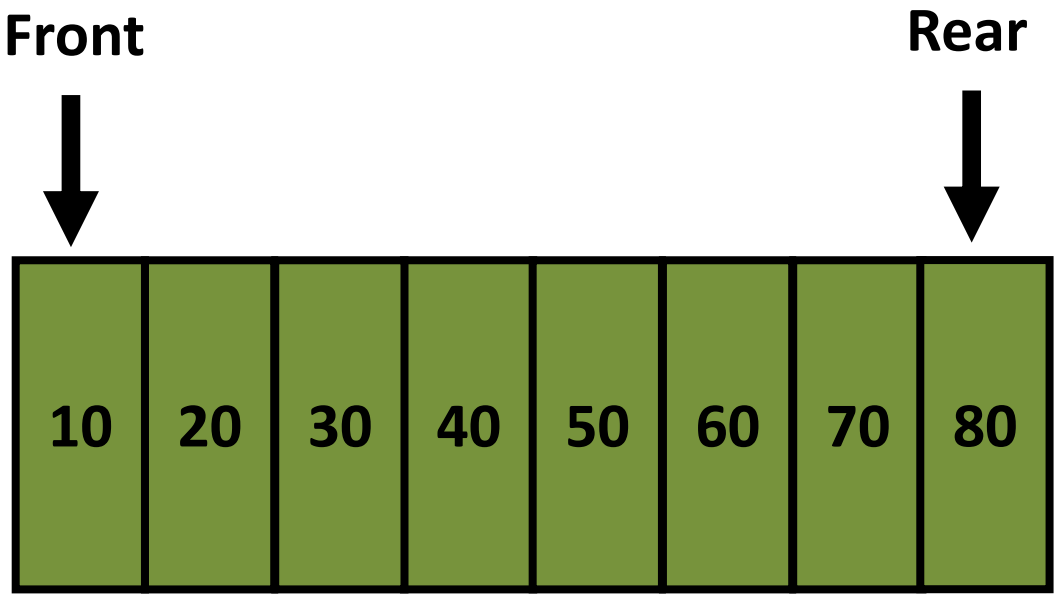
Queue Operations

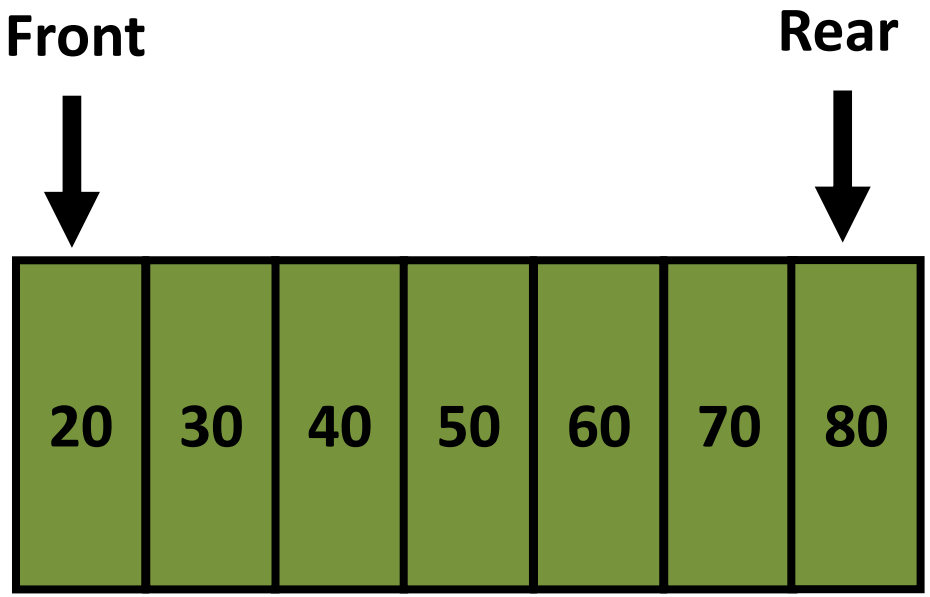
- Main Operations
  - Enqueue (Insert)
  - Dequeue (Remove)
  - Peek
- Other methods
  - Is Empty
  - Is Full
    - Is Full is applicable only for array-based implementation

Enqueue (Insert) Operation

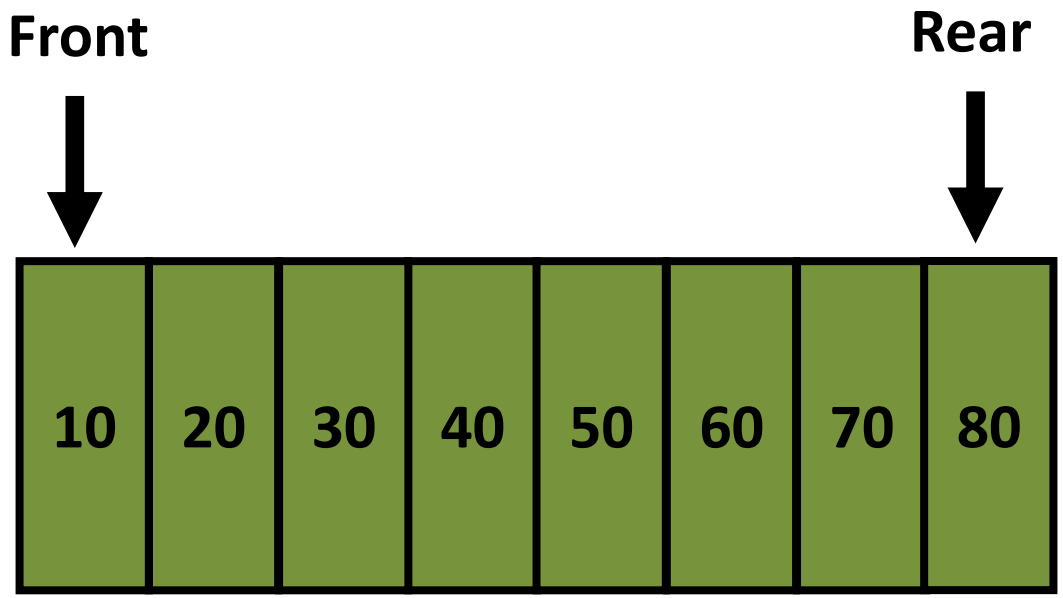


Dequeue (Remove) Operation





Peek Operation



Implementation of Queue

- Array based queue
- Linked list-based queue

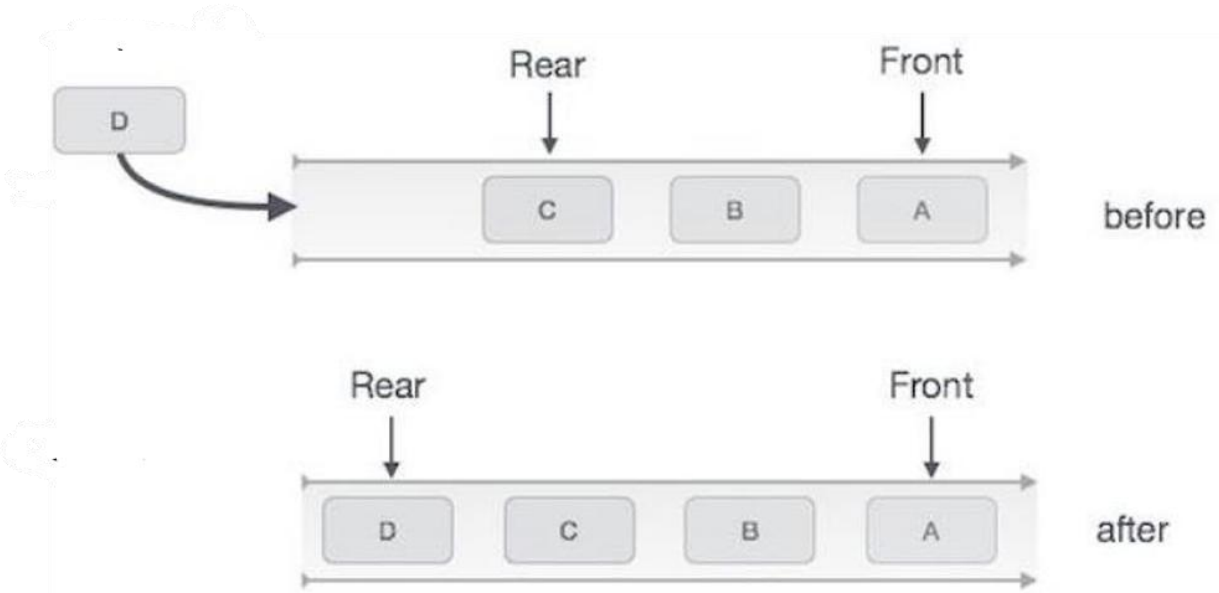
Array Based Queue Implementation

- **Linear queue**
  - Front or rear can get only up to the last (length – 1) element.
  - Maximum number of enqueue is same as the array size.
  - Less effective use of memory.
  - Rarely used.
- **Circular queue**
  - Both front and rear can wrap around like a circle.
  - At any time, the queue can keep maximum total number of elements which is the size of the array.
    - i.e. When elements are getting dequeue, those memory is available for new elements for enqueue.
  - More effective use of memory.

Linear Queue

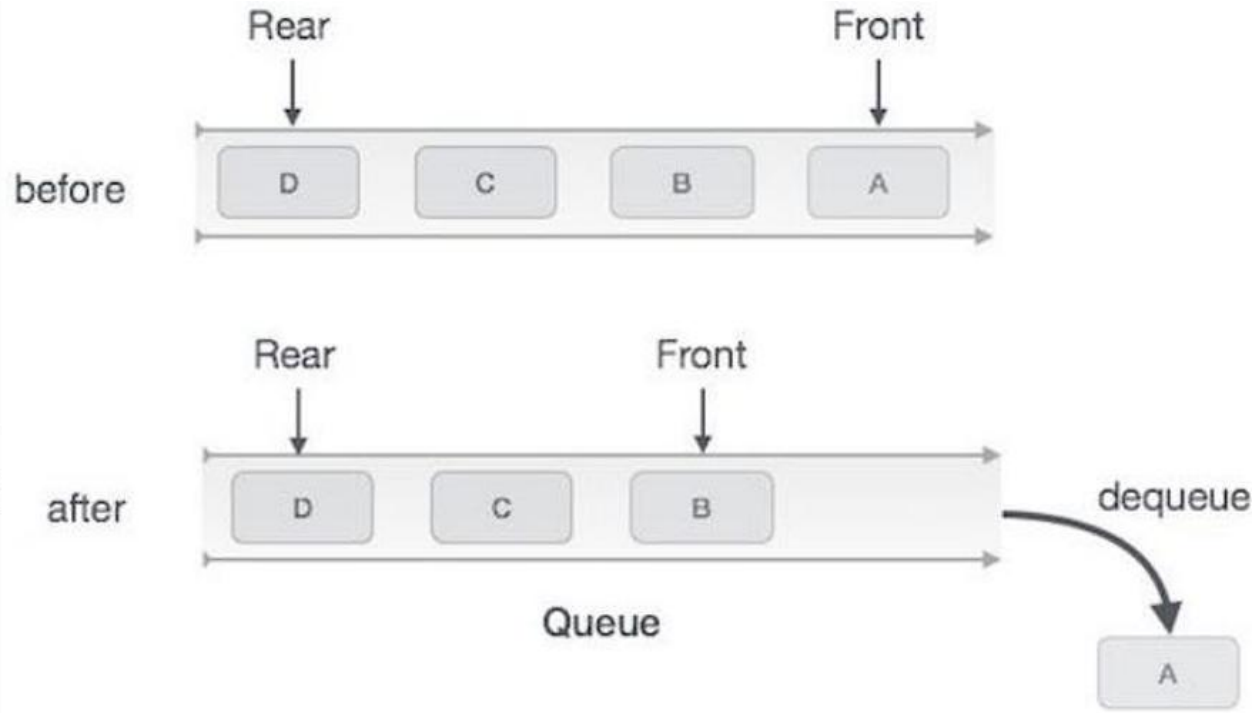
Enqueue - Operation

```
BEGIN enqueue(data)
   IF queue is full
      RETURN overflow
   ENDIF
   rear ← rear + 1
   queue[rear] ← data
   RETURN true
END PROCEDURE
```



Dequeue - Operation

```
BEGIN dequeue
   IF queue is empty
      RETURN underflow
   ENDIF
   data = queue[front]
   front ← front + 1
   RETURN true
END PROCEDURE
```



Circular Queue

Circular Queue Implementation – Array Based (Fixed Size)

- Use fixed size array.
- Keep track of indices of rear & front.
- Keep track of number of elements in the queue.
- Wrap around the rear index and front index to the left end (original initialized values) when those are at the right end.

Array Based Queue Implementation - Overview

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
  void enqueue(Object d) {
    // TODO
  }
  ….
```

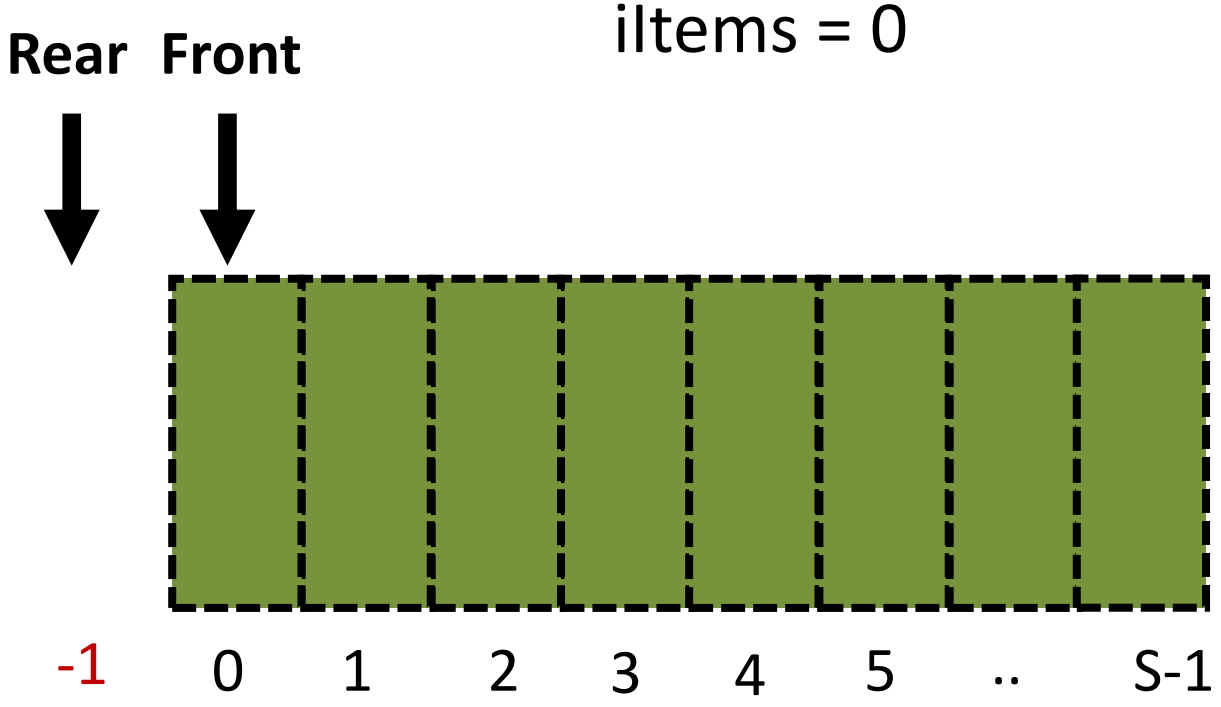
```
  …
  Object dequeue() {
     // TODO
  }
  Object peek() {
     // TODO
  }
  boolean isEmpty() {
     // TODO
  }
  boolean isFull() {
    // TODO
  }
}
```

Array Based Queue Implementation – Initial State

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
  ….
```

size = s

iItems = 0



isEmpty() Operation

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
    ….
```

```
boolean isEmpty() {
  return nItems == 0;
}
```

isFull() Operation

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
    ….
```

```
boolean isFull() {
  return nItems == size;
}
```

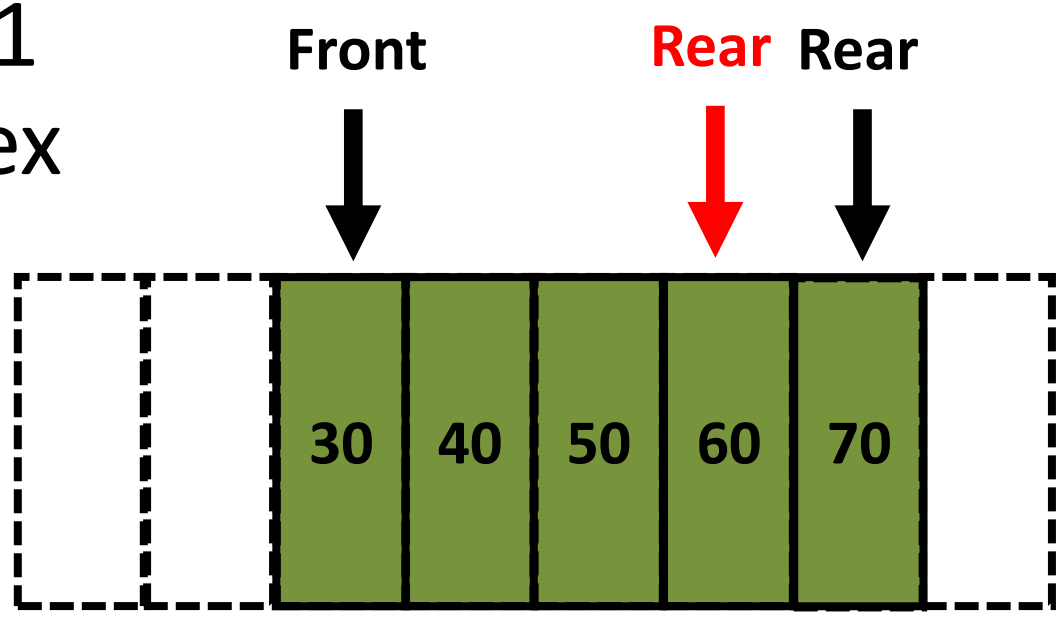
Enqueue (Insert) Operation

If queue is not full

- If rear element is the last element (index = size -1)
  - Wrap around to start (Set rear = -1)
- Increment rear index by 1
- Set data to new rear index
- Increment count by 1

Else

- Give overflow error



enqueue() Operation

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
    ….
```

```
void enqueue(Object d) {
  if (!isFull()) {
    if (rear == size - 1) {
      rear = -1;
    }
    data[++rear] = d;
    nItems++;
  } else {
    System.out.println("Queue
                        overflow" + d);
  }
}
```

Performance - O(1)

Dequeue (Remove) Operation

If queue is not empty

Assign value in the front index to a temp variable.

Increment front variable by one.

If front is equal to size

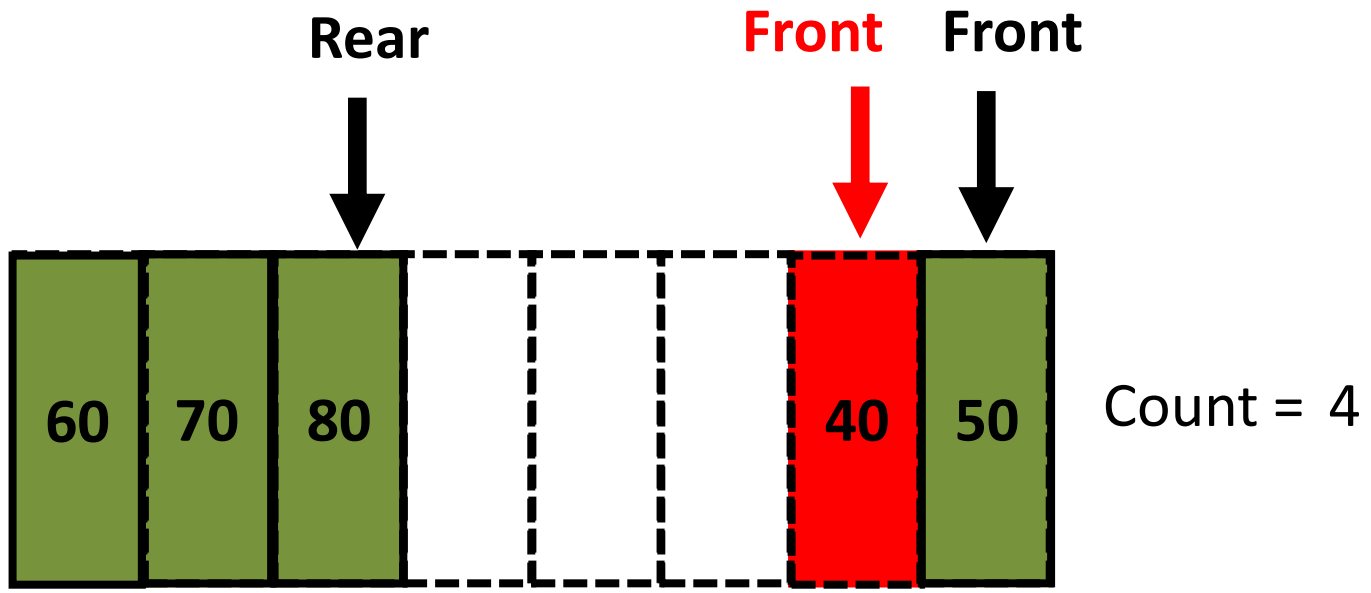
Set front = 0

Decrement count by 1

Return temp

Else

Underflow error



dequeue() Operation

```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
    ….
```

```
Object dequeue() {
  Object temp = null;
  if (!isEmpty()) {
    temp = data[front++];
    if (front == size) {
      front = 0;
    }
    nItems-;
  } else {
   System.out.println("Queue underflow.");
  }
  return temp;
}
```

Performance - O(1)

peek() Operation

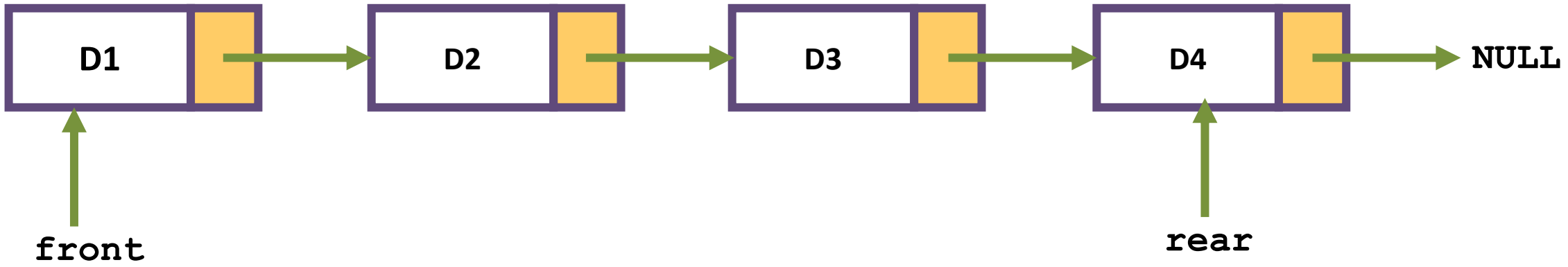
```
class ArrayQueue {
  Object[] data;
  int size,nItems,rear,front;
  ArrayQueue(int s) {
    size = s;
    nItems = 0;
    rear = -1;
    front = 0;
    data = new Object[s];
  }
    ….
```

```
Object peek() {
  Object t = null;
  if (!isEmpty()) {
    t = data[front];
  } else {
   System.out.println("Queue Underflow");
  }
  return t;
}
```

Linked List Based Queue Implementation

Queue – Linked List Based Implementation

- Implemented as a single linked list.
- Keep track of **`front`** and **`rear`**.
- **`Enqueue`** to the **`rear`** and **`dequeue`** from the **`front`**.

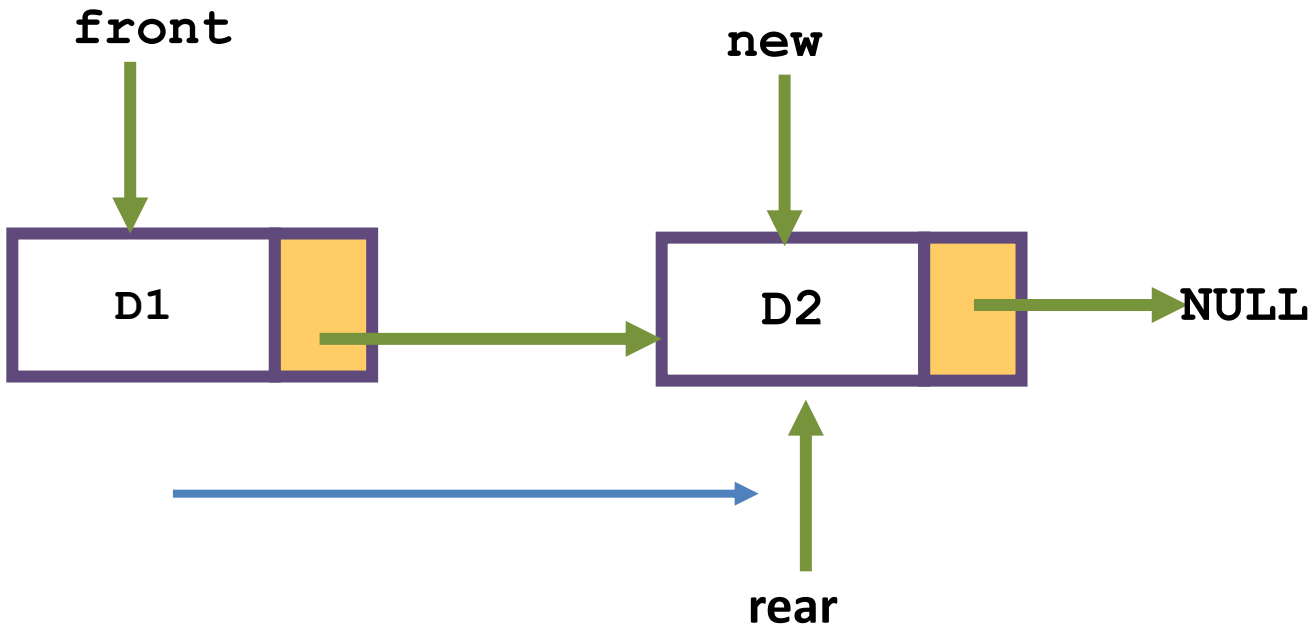


```java
public class Node {
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

```java
public class QueueUsingLinkedList {
    private Node front=null, rear = null;

    public boolean isEmpty() {
            // TODO
    }

    public void enqueue(int value) {
            // TODO
    }

    public int dequeue() {
            // TODO
    }

    public int peek() {
            // TODO
    }

    public void display() {
           // TODO
    }
}
```

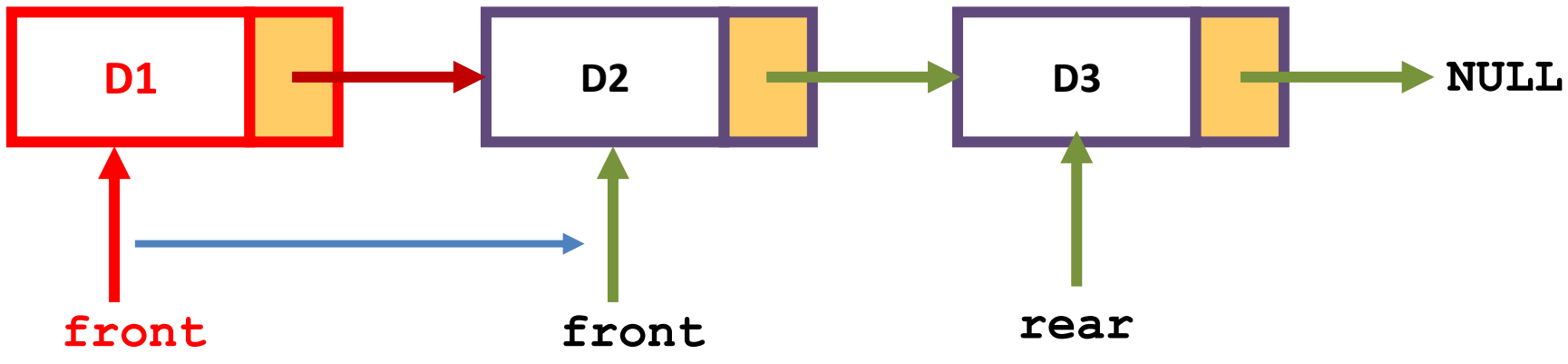
EnQueue

```
public void enqueue(int value) {
   Node newNode = new Node(value);
   if (rear == null) {
      front = rear = newNode;
      return;
   }
    rear.next = newNode;
    rear = newNode;
}
```



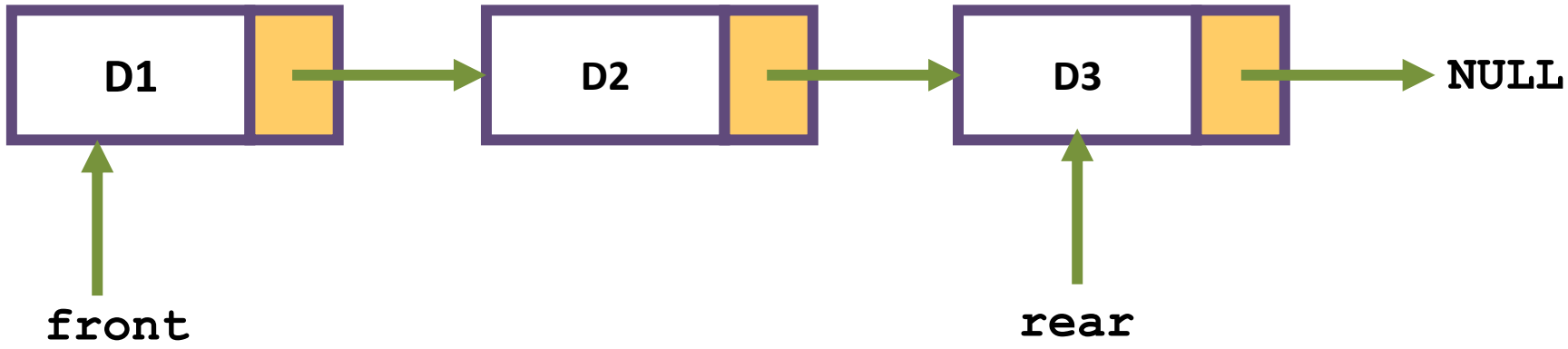
DeQueue

```
public int dequeue() {
   if (isEmpty()) {
      System.out.println("Queue is empty!");
      return -1;
   }
   int value = front.data;
   front = front.next;
   if (front == null) {
      rear = null;
   }
   return value;
}
```



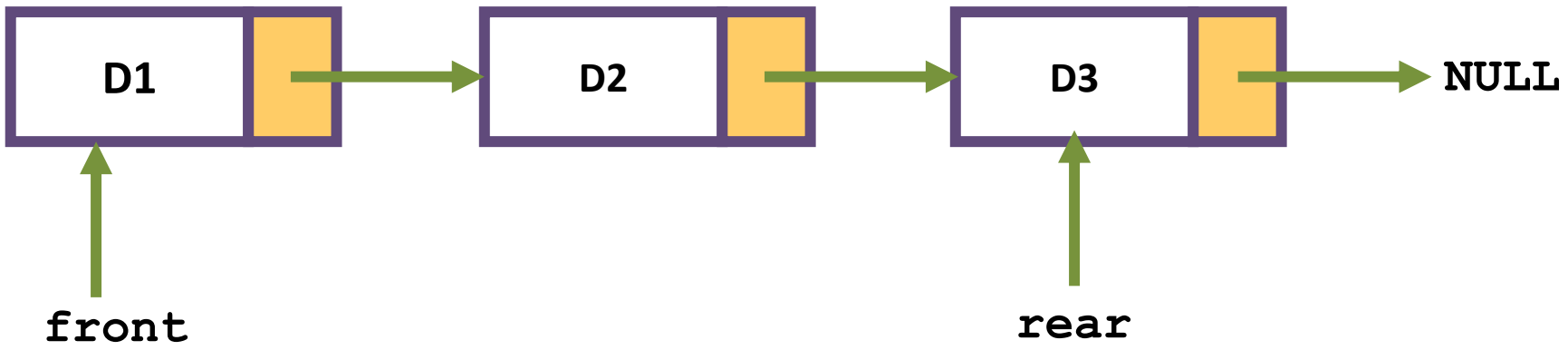
Peek

```
public int peek() {
   if (isEmpty()) {
      System.out.println("Queue is empty!");
      return -1;
   }
   return front.data;
}
```

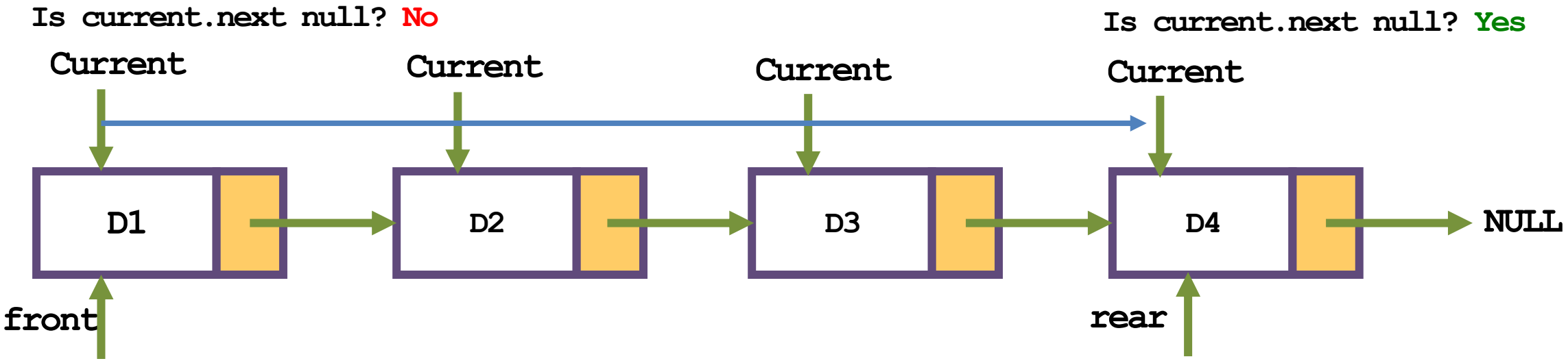


isEmpty

```
public boolean isEmpty() {
   return front == null;
}
```



```
public void display() {
   Node current = front;
   System.out.print("Queue: ");
   while (current != null) {
      System.out.print(current.data + " ");
      current = current.next;
   }
   System.out.println();
}
```



Priority Queue

Priority Queue

- Enqueue to the proper priority position.
  - Keep a field to identify the priority.
  - Performance - O(n)
- Dequeue from the front.
  - Performance - O(1)

Note – Only the enqueue operation must be changed.

Exercise – Implement the priority queue

Past Papers

2022 #1-c

If one evaluates the following pseudocode algorithm segment using a dry run, what would be the final content of the queue?

```
q = queue();
for(j=1; j<6;j++)
{
   q.enqueue (j);
}
for(i=1;i<6;i++)
{
   if(i%2 <>  0)
   {
   q.dequeue()
   }
   else{
   q.enqueue(i*3-1)
   }
}
```

ANSWER IN THIS BOX

contents of the queue =4,5,5,11

2021 #1-b-iii

A circular queue can be implemented for the following scenarios.

- Memory management
- CPU Scheduling:
- Traffic system

Discuss why a circular queue is suitable for implementing the above scenarios than any other data structure.

Answer

- Memory management: in case of a circular queue, the memory is managed efficiently by placing the elements in a location which is unused.
- CPU Scheduling: The operating system uses the circular queue to insert the processes and then execute them.
- Traffic system: . Each light of traffic light gets ON one by one after every interval of time. Like red light gets ON for one minute then yellow light for one minute and then green light. After green light, the red light gets ON.

2021 #1-b-iv

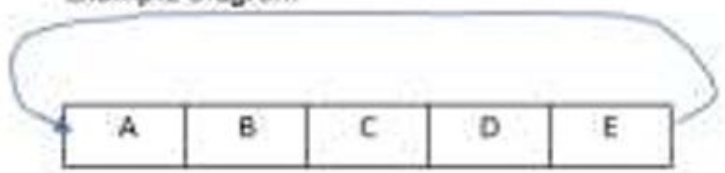
Consider the following algorithm, which can be used to insert an element to a circular queue.

```
STEP 1: if (Rear+1) % Max =Front
          Write("Overflow")
          Goto STEP 4
        Endif
STEP 2: if Front=-1 and Rear=-1
           Set Front=Rear=0
        Eles if Rear=Max-1 and Front! = 0
                Set Rear =0
             Else
                Set Rear=(Rear+1) % Max
        Endif
STEP 3: Set Queue (Rear)=Val
STEP 4: Exit
```

Using a suitable diagram, describe how each step of the algorithm works.

ANSWER IN THIS BOX

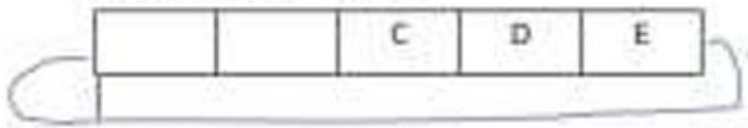
Example Diagram



One wants to new element F

Step 1: Queue is full, unable to insert, go to Step 4 and Exit

Step 2: 1st Part - Insert F



After the insertion

F		C	D	E

Step 2: 2nd Part - Insert F

C	D	E		

After the insertion

C	D	E	F	

2019 #1-c

Write pseudocode algorithms/Java code to implement two basic queue operations Enqueue and Dequeue **using a maximum of two stacks for each**.

ANSWER IN THIS BOX

enQueue(q, x):

Step 1: While stack1 is not empty, push everything from stack1 to stack2.
Step 2:Push x to stack1 (assuming size of stacks is unlimited).
Step 3: Push everything back to stack1.

Or equivalent

DeQueue(q):

Step 1:If stack1 is empty then error
Step 2: Pop an item from stack1 and return it

Or equivalent

Thank You

Aurora Computer Studies | auroracs.lk